

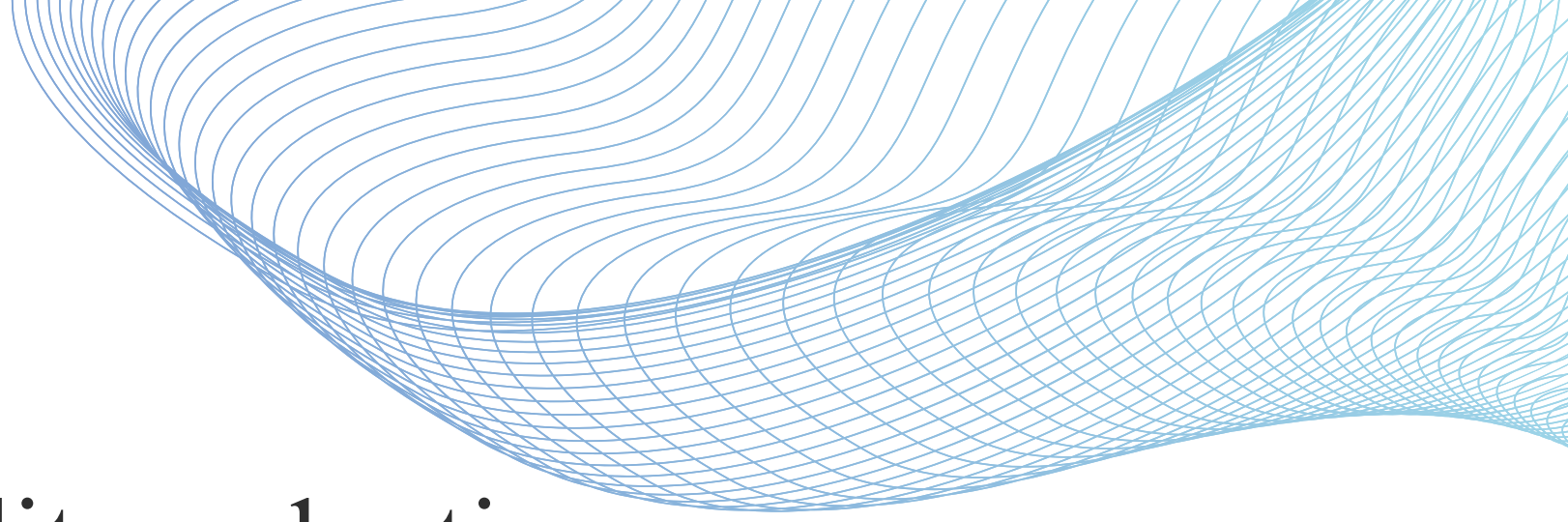


PREDICTIVE DATA ANALYTICS

LESSON 3: DIMENSIONALITY REDUCTION

nguyenhau@vnu.edu.vn

DIMENSIONALITY REDUCTION



There are two components of dimensionality reduction:

- **Feature selection:** In this, we try to find a subset of the original set of variables, or features, to get a smaller subset which can be used to model the problem.
- **Feature extraction:** This reduces the data in a high dimensional space to a lower dimension space, i.e. a space with lesser no. of dimensions.

principal component analysis (PCA), singular value decomposition (SVD), and linear discriminant analysis (LDA)

FEATURE SELECTION METHODS

Filter Methods

1. Information Gain
2. Chi-square Test
3. Fisher's Score
4. Correlation Coefficient
5. Variance Threshold
6. Mean Absolute Difference
7. Dispersion Ratio

Wrapper Methods

1. Forward Feature Selection
2. Backward Feature Elimination
3. Exhaustive Feature Selection

Embedded Methods

1. LASSO Regularization (L1)
2. Random Forest Importance
- 3.

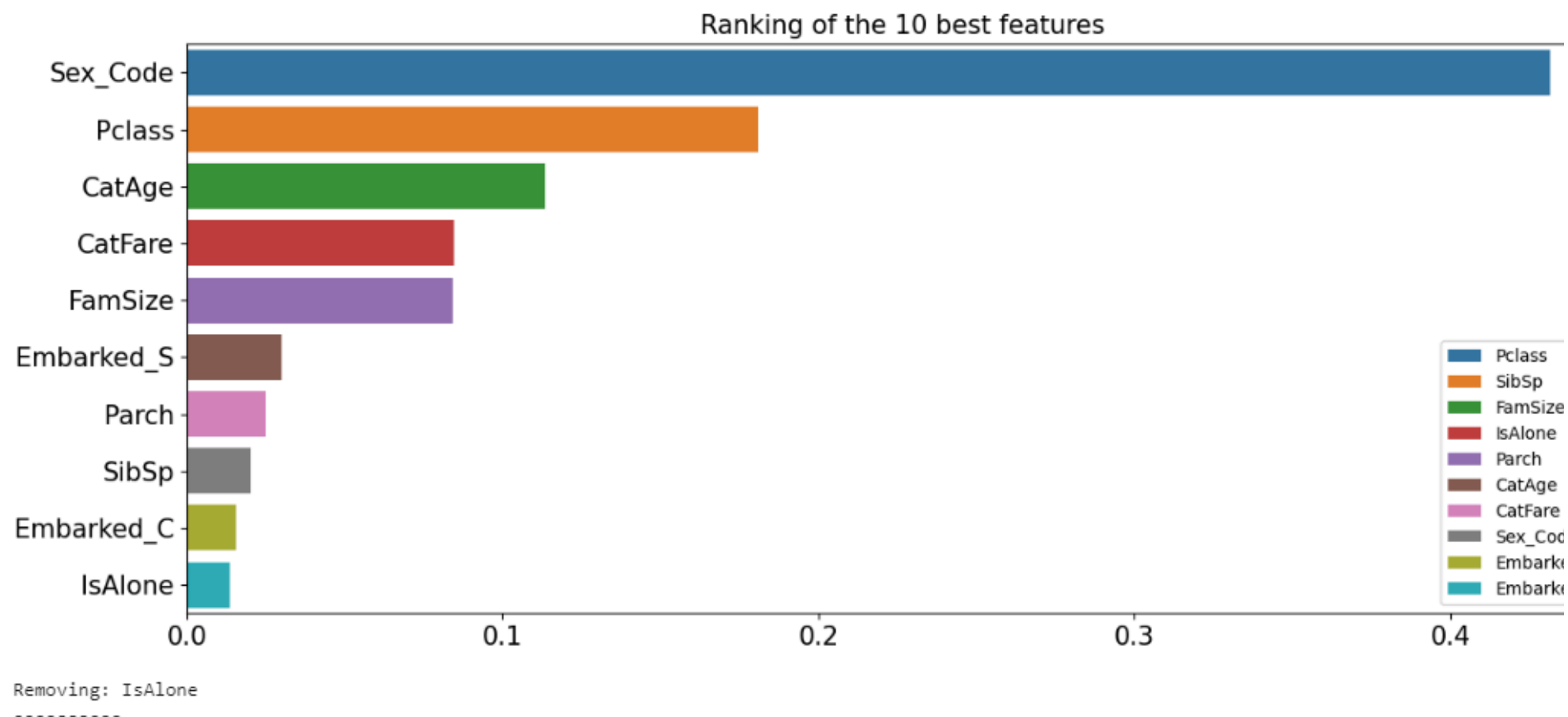
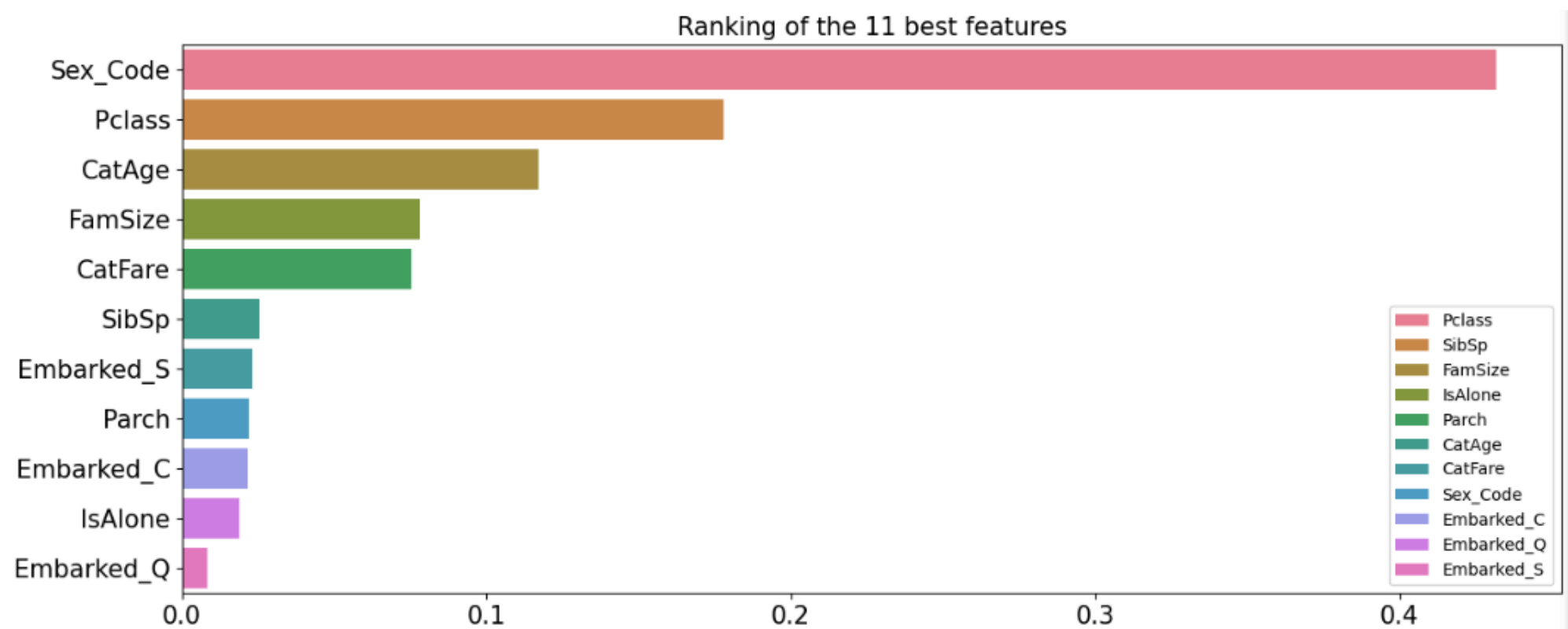
<https://www.analyticsvidhya.com/blog/2020/10/feature-selection-techniques-in-machine-learning/>

1. RECURSIVE FEATURE ELIMINATION

The RFE method iteratively identifies the most important features in a dataset by removing the least important feature in each iteration. The steps are:

1. Fit your model on the dataset
2. Call the feature importance ranking
3. Remove the lowest ranked feature
4. Repeat steps 1-3 until you reach desired number of "best" features

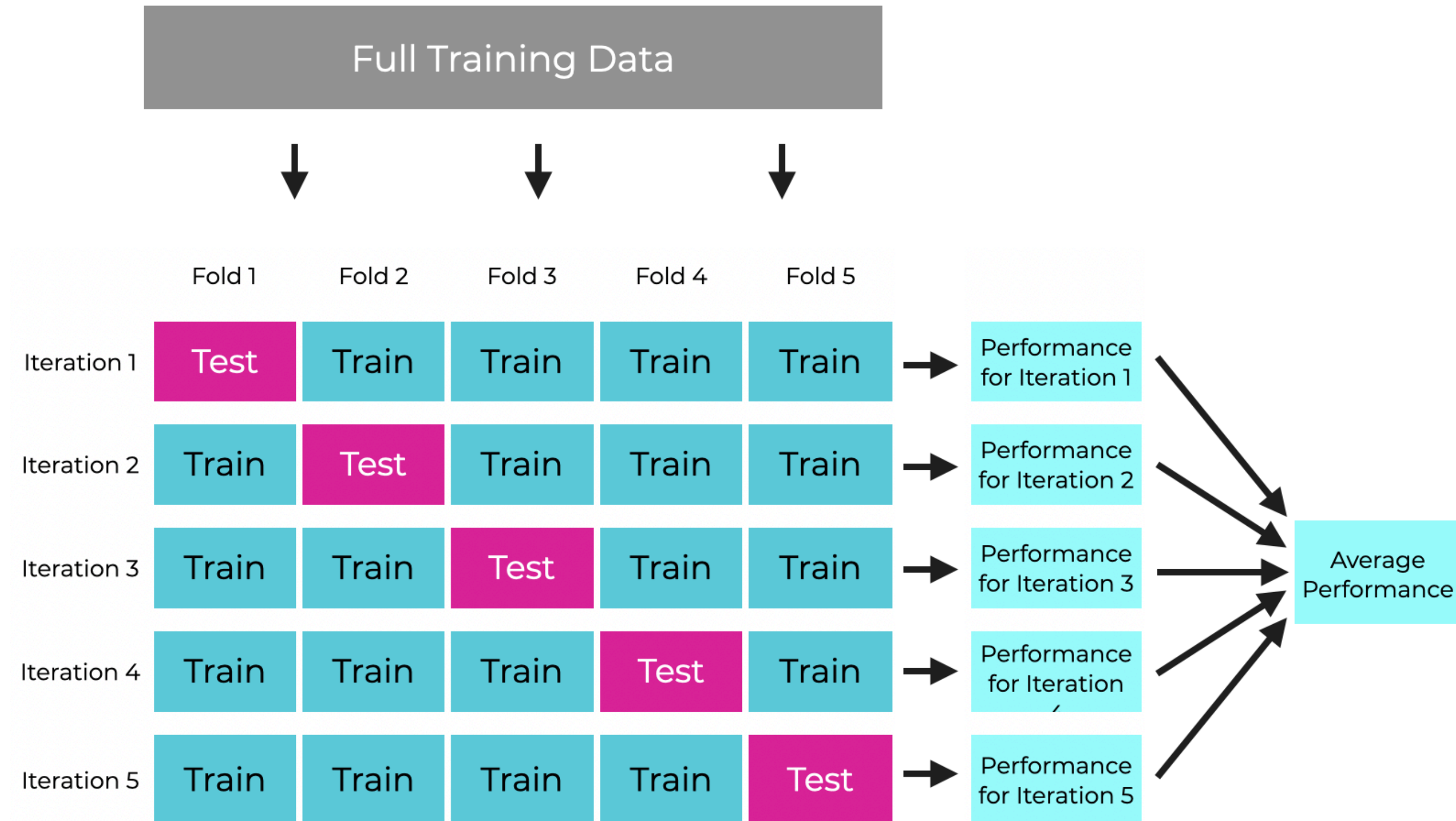
RECURSIVE FEATURE ELIMINATION



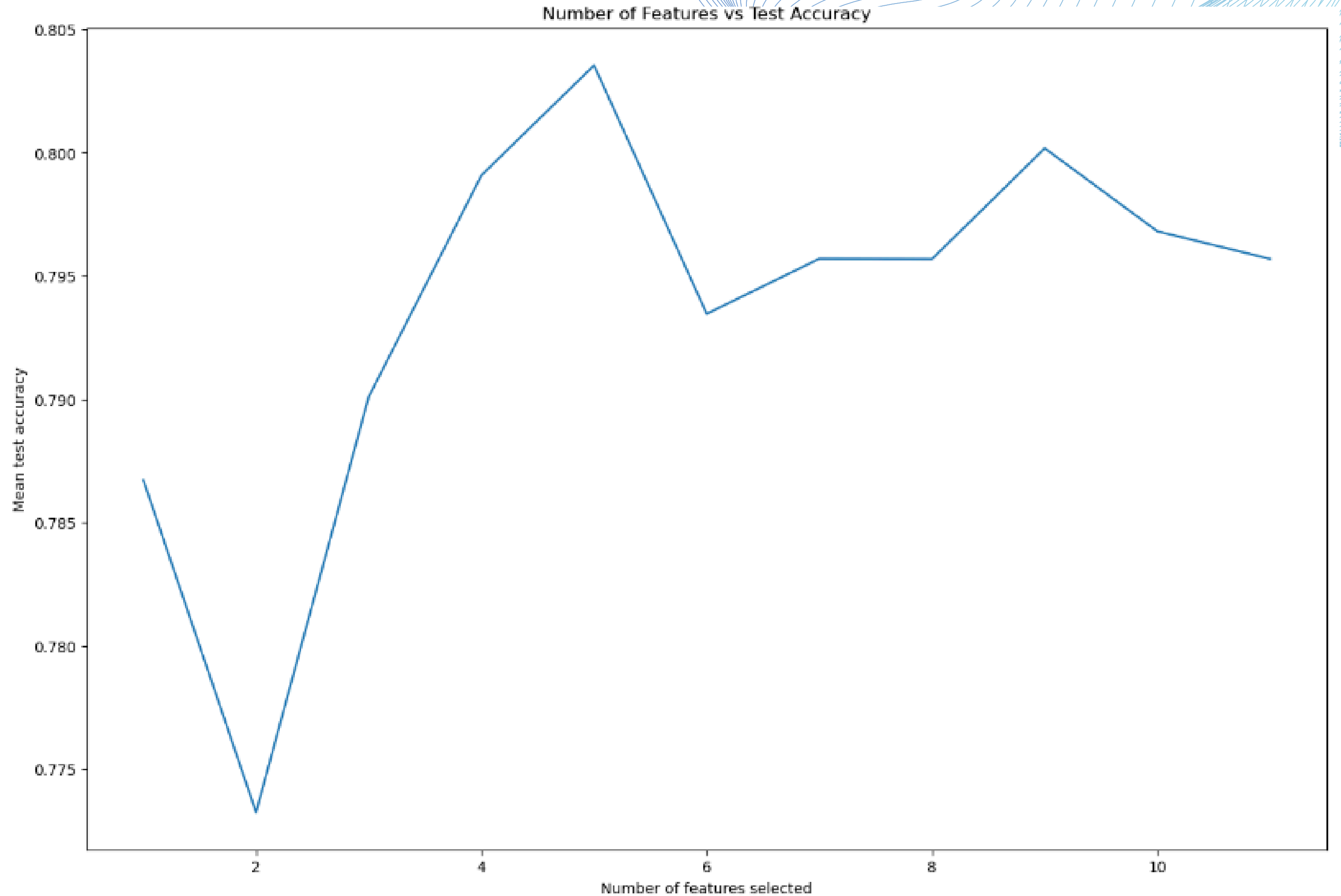
RFE CV

Recursive Feature Elimination with Cross-Validation (RFE CV) is the technique obtained when we combine **RFE and cross-validation**. By identifying the number of features that consistently improve model performance over multiple folds, RFE CV mitigates overfitting concerns.

K-FOLD CROSS VALIDATION



RFECV



RFECV VIA SCIKIT-LEARN

```
from sklearn.feature_selection import RFECV
from sklearn.model_selection import StratifiedKFold

cv_split = StratifiedKFold(5, shuffle=True, random_state=0)

rfecv = RFECV(
    estimator=model,
    step=1,
    min_features_to_select=1,
    cv=cv_split,
    scoring="accuracy",
    n_jobs=-1,
)

rfecv.fit(X, y)

print(f"Optimal number of features: {rfecv.n_features_}")
print(f"Optimal features: {list(rfecv.get_feature_names_out())}")
```

NOTES ON RFECV

- **Reliability of feature importance scores:** Feature importance scores quantify the extent to which a model relies on a feature to make predictions. They do not necessarily quantify the contribution of a feature to the overall accuracy of a model (i.e. the feature's usefulness). This is a subtle but fundamental difference, that shows RFECV won't magically fix a poor base model. Prior to inspecting the feature importances, it is important to check that the model predictive performance is high enough as there would be little interest of inspecting the important features of a non-predictive model.
- **Drawbacks of RFECV:** The limitations of RFECV stem from the limitations of the feature importance measures it relies on. For instance, decision trees, which are often used in feature importance calculations, struggle with high cardinality features.

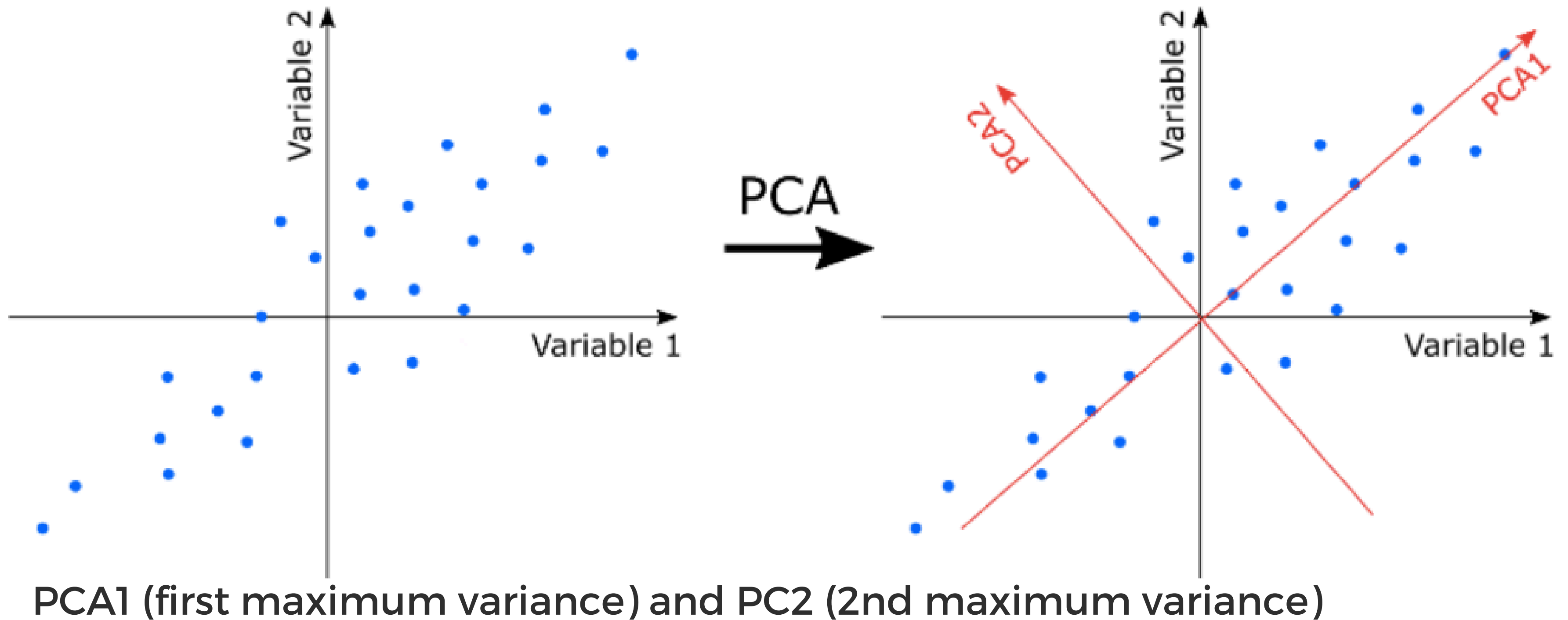
NOTES ON RFECV

- **How to handle multicollinear/correlated features:** The RFECV technique can handle correlated features but does not necessarily eliminate highly correlated features, since we have seen it be able to select highly correlated variables too. Instead, we should take caution about how well suited your model is to handling highly correlated variables, i.e. if your base model can handle correlated features, then RFECV itself will likely not present any problems either. *If your model requires less degree of multicollinearity in the data, consider applying another dimensionality reduction technique such as PCA before using RFECV.* PCA removes the highly correlated features from the dataset and makes RFECV more reliable as a second feature selection layer.
- **Choose an appropriate RFECV step size**

2. PCA (PRINCIPAL COMPONENT ANALYSIS)

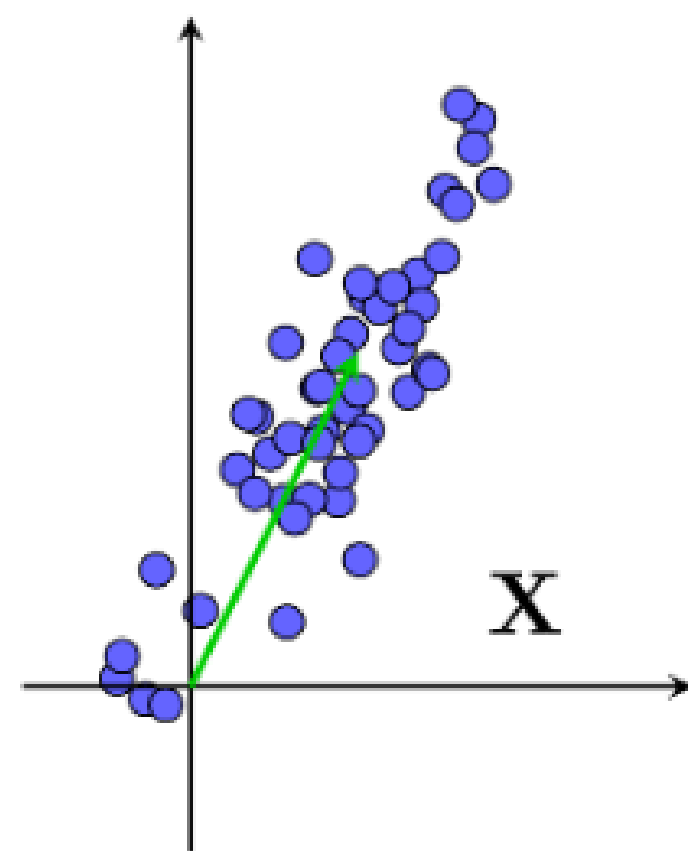
- Principal component analysis (PCA) is a is an **unsupervised linear transformation** technique which is primarily used for feature extraction and dimensionality reduction.
- It aims to find **the directions of maximum variance** in high-dimensional data and projects the data onto a new subspace with equal or fewer dimensions than the original one.
- The way PCA is different from other feature selection techniques like random forest, regularization techniques, forward/backward selection techniques... is that **it does not require class labels** to be present (thus called as unsupervised)

PCA (PRINCIPAL COMPONENT ANALYSIS)

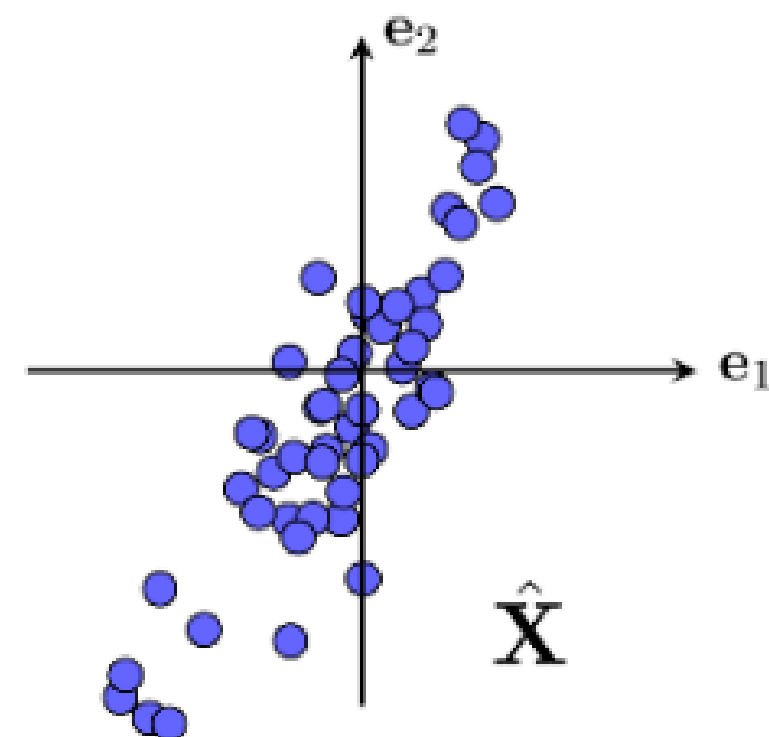


PCA PROCEDURE

1. Find mean vector



2. Subtract mean

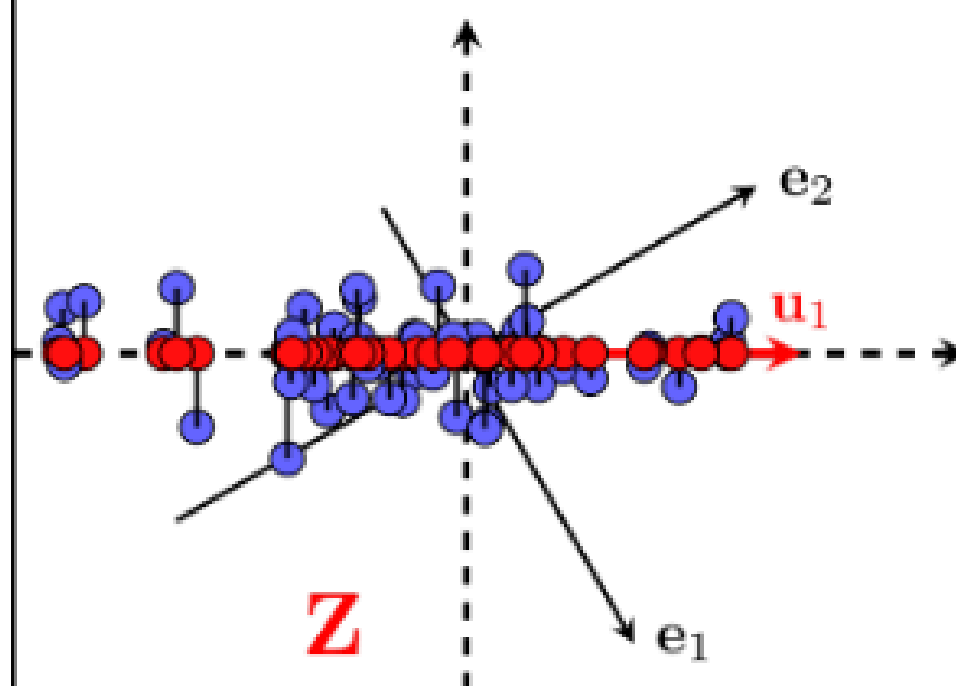


3. Compute covariance matrix:

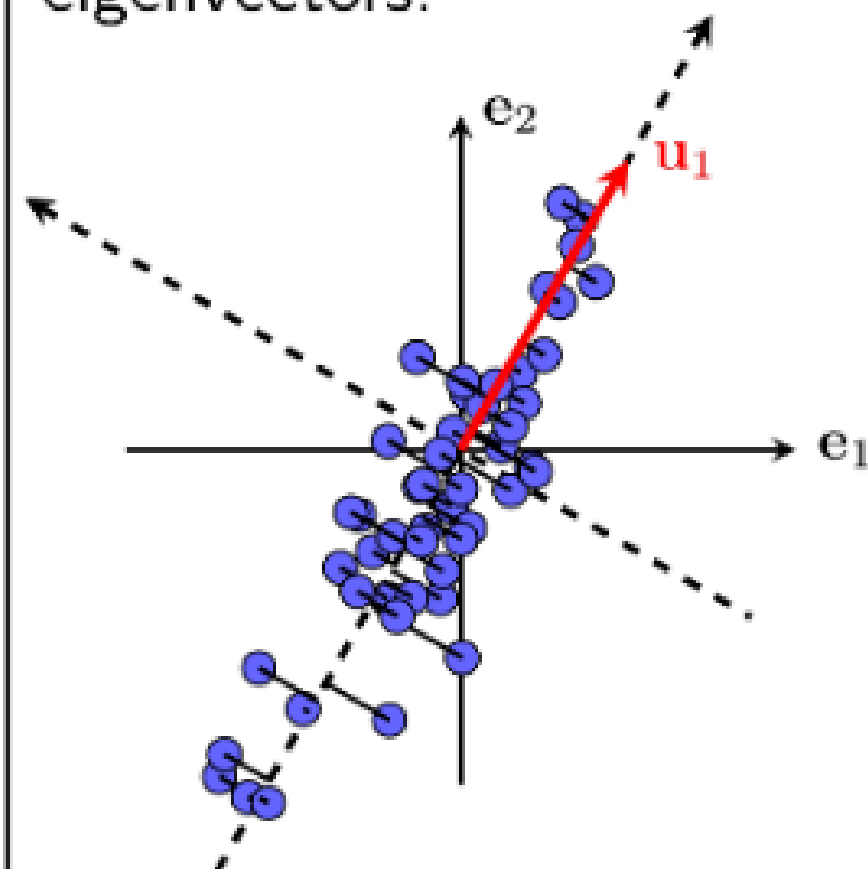
$$\mathbf{S} = \frac{1}{N} \hat{\mathbf{X}} \hat{\mathbf{X}}^T$$

4. Compute eigenvalues and eigenvectors of $\mathbf{S}$:
 $(\lambda_1, \mathbf{u}_1), \dots, (\lambda_D, \mathbf{u}_D)$
 Remember the orthonormality of $\mathbf{u}_i$.

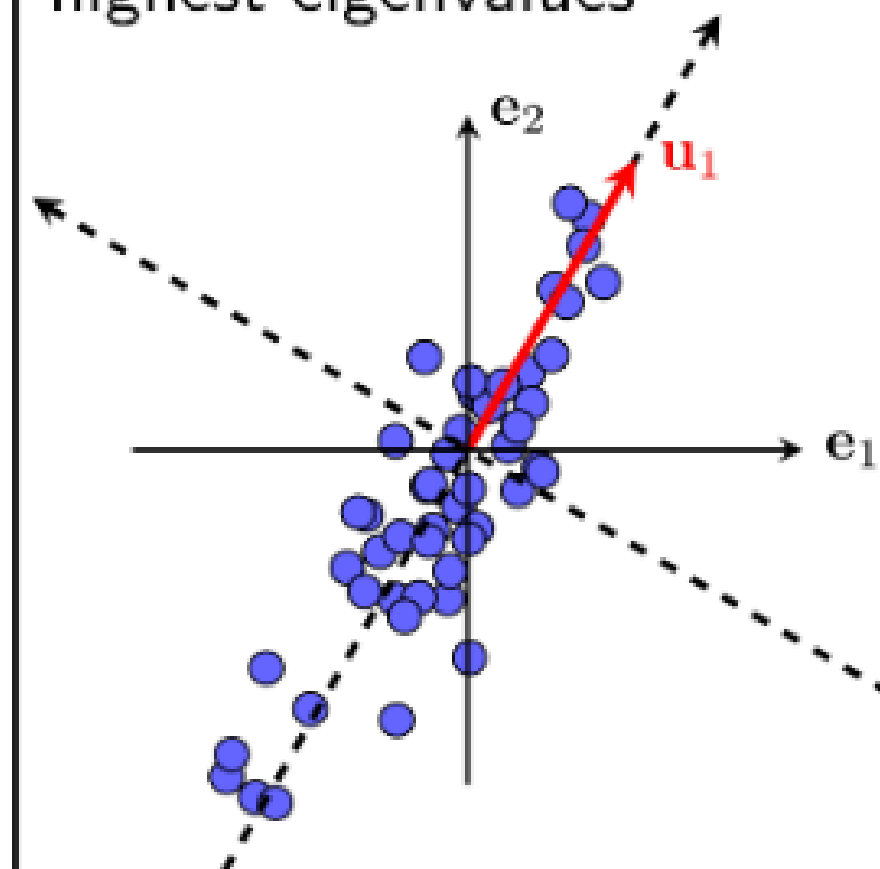
7. Obtain projected points in low dimension.



6. Project data to selected eigenvectors.



5. Pick K eigenvectors w. highest eigenvalues



PCA ALGORITHM FOR FEATURE EXTRACTION

1. **Standardize the dataset:** the standard deviation of all features / variables get measured on the same scale
2. **Construct the covariance matrix:** create $n \times n$ -dimensional covariance matrix, where n is the number of dimensions in the dataset
3. **Perform Eigen decomposition of covariance matrix**
4. **Selection of most important Eigenvectors / Eigenvalues**
5. **Projection matrix creation of important eigenvectors**
6. **Training / test dataset transformation**

HOMEWORK

- Apply all these techniques to any dataset using R

THANK YOU

